

Overleaf Viewer: Technical Documentation & Developer Guide

Introduction

Overleaf Viewer is a utility application designed to solve the challenge of accessing stable PDF URLs for public Overleaf projects. By default, Overleaf does not provide static endpoints for the latest compiled versions of projects. Overleaf Viewer functions as a proxy service, interacting with Overleaf's internal authentication and compilation workflows to provide users with a stable link that consistently resolves to the most recent version of a project's PDF.

Architecture Overview

The application is built on the **Next.js** framework, leveraging a hybrid architecture of server-side API routes and a client-side interface.

- **Client-Side:** A React-based interface built with TailwindCSS allows users to input Overleaf share URLs, parse project tokens, and generate stable links.
- **Server-Side:** Next.js API routes act as the backend engine. These routes manage session authentication, trigger Overleaf project compilation, and fetch the resulting PDF files.
- **Proxy Logic:** The system mimics user interactions with Overleaf to perform the necessary background tasks required to retrieve an up-to-date PDF, bridging the gap between Overleaf's dynamic interface and a static URL output.

Setup and Installation

Prerequisites

- Node.js (LTS version recommended)
- npm or yarn package manager

Configuration

1. Clone the repository to your local machine.
2. Install dependencies: `bash npm install`
3. Configure environment variables as required by the Overleaf authentication proxy logic.
4. Run the development server: `bash npm run dev`
5. Access the application at `http://localhost:3000`.

Core Components

Frontend UI (`app/page.tsx`)

The primary entry point for users. It provides a clean, TailwindCSS-styled interface where users submit an Overleaf share URL. The UI parses the URL to extract the unique project token and facilitates the generation and copying of the resulting stable link.

PDF Viewer (`app/view/[token]/page.js`)

This component is responsible for the presentation layer of the compiled document. It takes the project token and renders the finalized PDF within an iframe, providing a seamless viewing and download experience for the end user.

Overleaf Handler (`app/api/overleaf/[token]/route.js`)

The core integration module. This route contains the logic for:

- * Interfacing with Overleaf's backend.
- * Handling session authentication (including CSRF and cookies).
- * Triggering remote project compilation.
- * Returning the finalized PDF link to the client.

Caching Strategy

To optimize performance and reduce the load on both the Overleaf platform and the application server, the project utilizes **NodeCache**.

- **Session Metadata:** Authentication cookies and CSRF tokens are cached to prevent redundant handshake requests.
- **PDF Delivery:** Finalized PDF links are stored in an in-memory cache to ensure that repeat requests for the same project receive rapid responses without re-triggering the

compilation workflow.

API Reference

GET /api/overleaf/[token]/

Handles the request for a specific project's PDF.

Parameters:

- `token` (string): The unique identifier parsed from an Overleaf share URL.

Behavior:

- Checks the `NodeCache` for existing valid links or session data.
- If unavailable, initiates the handshake and compilation process via the Overleaf internal workflow.
- Returns the stable PDF link.
- **Error Handling:** Returns standard HTTP error codes (e.g., 404 for invalid tokens, 500 for compilation failures) with descriptive messages to assist in debugging.

Contribution Guidelines

We welcome contributions to improve the reliability and functionality of Overleaf Viewer.

- **Coding Standards:** The project is written in **TypeScript**. Please ensure all new code maintains strict type safety.
- **Linting:** The project uses **ESLint** to enforce code quality. Run `npm run lint` before submitting a pull request to ensure adherence to project standards.
- **Submission:** Submit all changes via Pull Requests on the repository, ensuring that your code is documented and tested against the existing API logic.